VOCABULAIRE ET JARGONS

Démystifier les principaux concepts du Craft

INTRODUCTION

- Julien
  - DevOps
  - Python
  - passions : informatique et Craft
  - talker
  - bénévole aujourd'hui
- tutoiement, interventions et échanges

LE SUJET

Chaque métier a son jargon, chaque spécialité en rajoute. Cela rend la communication efficace, une fois qu'on maitrise tous ces termes.

Ensemble, faisons une passe sur le vocabulaire lié au Craft, en toute simplicité, pour découvrir ces termes parfois inconnus, parfois déjà connus sous un autre nom.

LES ATTENDUS ?

Quels sont les termes à définir ?

COMMENT DÉFINIR ?

C'est compliqué !

1. *Fait de déterminer les caractéristiques d'un concept, d'un mot, d'un objet, etc., ensemble des propriétés essentielles de quelque chose. Exemple : "La définition de ce mot n'est pas facile."*

— Larousse

1. *Opération par laquelle on détermine le contenu d'un concept en énumérant ses caractères.*

— Le Robert

Jeu : quelles différences ?

DEFINE DEFINE

TodePond - Define Define

A definition is a tool that we use to express ourselves, and to communicate. Sometimes, the definition is for other people. Sometimes it's just for you.
The definition that you use tell us something about you.
Personally, I love to use definitions very loosely. But what about you ?
What definitions do you use ?

Une définition est un outil que l'on utilise pour s'exprimer, et pour communiquer. Des fois, la définition est pour les autres personnes. Des fois, juste pour soi. La définition que tu utilises nous dis quelque chose de toi. Personnellement, j'aime utiliser les définitions de façon très approximative. Et toi ? Quelles définitions tu utilises ?

MÉTHODE

- exercice ensemble
- je suis préparé
- pas de conviction
- objectif : se familiariser avec ces termes

N°1

```python
import requests

def récupérer_le_bundle():
    url = "http://toto-ftp.service.corp.com"
    response = requests.get(url)
    print("received bundle")
    return response.json()

```

Quel terme j'essaye de vous faire deviner ?

CODE REVIEW

Quelle définition vous proposeriez ?

Code Review : examen minutieux du code produit par une personne, afin d'en déceler des erreurs potentielles ou factuelles, pour faire des retours constructifs, pouvant mener à des discussions, afin d'améliorer la qualité, de partager des pratiques, d'apprendre et de construire une culture d'équipe

N°2

```
grid_size_in_meters = 200
length_of_degree_of_arc_in_france_in_kilometers = 111.358
step = grid_size_in_meters / length_of_degree_of_arc_in_france_in_kilometers
```

Où est le bug ?

Quelle est la cause ?

Quel terme j'essaye de vous faire deviner ?

CODE SMELL

Code smell : quelque chose qui en soit n'est peut-être pas une erreur, mais risque d'en causer plus tard.

Vous avez des exemples ?

Clean code !

N°3

```
def which_day_of_the_week(date: Date) -> WeekDay:
    # https://zestedesavoir.com/billets/3638/calculer-le-jour-de-la-semaine-de-nimporte-quelle-date/
    month_table = (0, 3, 3, 6, 1, 4, 6, 2, 5, 0, 3, 5)
    return WeekDay(
        (
            date.day
            + (date.year - 1900)
            + floor((date.year - 1900) / 4)
            + month_table[date.month - 1]
        ) % 7
    )
```

Est-ce que ma fonction est correcte ?

Comment en être sûr ?

Quel terme j'essaye de vous faire deviner ?

TEST

Quelle définition vous proposeriez ?

Test : contexte + action = résultat attendu ?
Autrement dit : s'assurer du fonctionnement du code tel que spécifié.
Pas forcément automatiquement, peut-être manuellement ou hybride.

Comment tester la fonction précédente ?

Comment on testerait récupérer_le_bundle ? On en discute ce midi !

N°4

```python
RESSOURCE_URL = "https://toto.service.corp.com"

def try_to_download_data_and_extract_payload() -> dict[str, Any] | None:
    # get the response
    try:
        response = requests.get(RESSOURCE_URL)
    except request.RequestError:
        logger.exception("failed to fetch the data")
        return None
    # check the return code
    if response.status != 200:
        logger.error(f"downloading data failed, {response.status=}")
        return None
    logger.debug("data downloaded successfully")
    # parse the response
    try:
        data = response.json()
    except JSONDecodeError:
        logger.exception("invalid downloaded data")
        return None
    else:
        return data
```

Est-ce que vous préférez ce code ?

Quel terme j'essaye de vous faire deviner ?

REFACTORING

Refactoring (version stricte): changer le fonctionnement/la structure interne, sans impacter le fonctionnement externe. Pour améliorer des propriétés liées à la qualité : lisibilité, évolutivité, ... Allant du simple renommage d'une variable locale, jusqu'à changer complètement toute l'architecture.

⚠️ Définition variable !

N°5

Qui aurait préféré des questions individuelles plutôt que collectives ?

Pourquoi préférer le faire collectivement ?

Quel terme j'essaye de vous faire deviner ?

PAIR PROGRAMMING

Pair Programming (version stricte) : il y a un "conducteur" (driver) qui a le clavier et la souris, et un "navigateur/observateur" qui dicte comment coder au conducteur. Les rôles sont échangés à intervalles réguliers.

⚠️ Définition variable !

Quels bénéfices ?

Mais là vous n'êtes pas que 2 ...

Quel terme j'essaye de vous faire deviner ?

MOB PROGRAMMING

Quelle définition vous proposeriez ?

Mob Programming : du pair programming, mais pas en paire, en trio, quatuor, quintette, sextuor, septuor, 8, 9, 10 ou plus … peu importe le nombre

En cours de renommage :

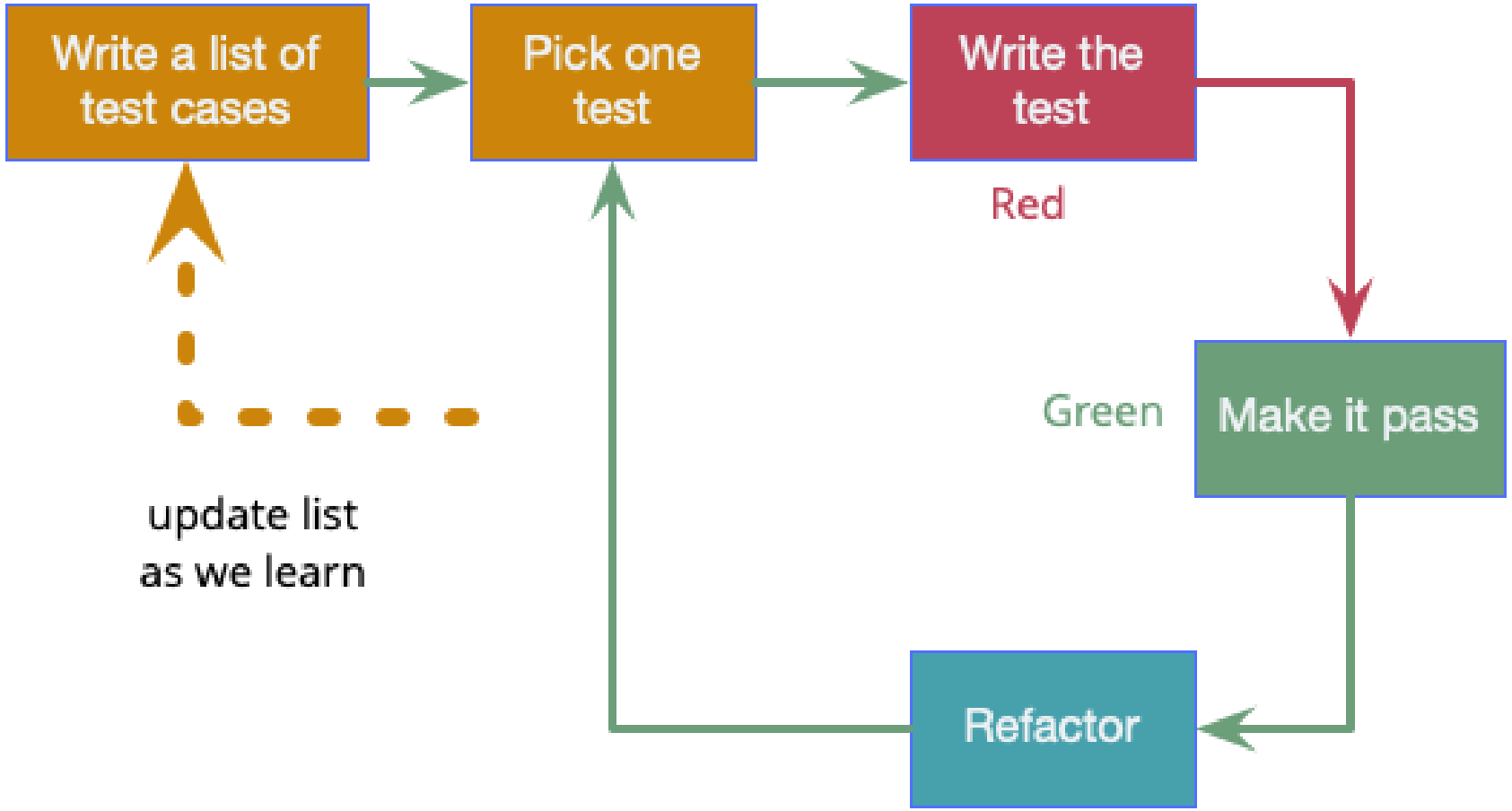
Software teaming : toute l'équipe (les devs bien sûr, mais aussi PO, UX, test, archi, cyber, …) réunie ensemble pour produire du logiciel. Toutes les personnes requises sont présentes simultanément, pour travailler toutes ensemble sans blocage.

N°6 : TDD

Le terme c'est : TDD.

Mais je dois d'abord vous parler de RGR.

LE RED/GREEN/REFACTOR DU TDD

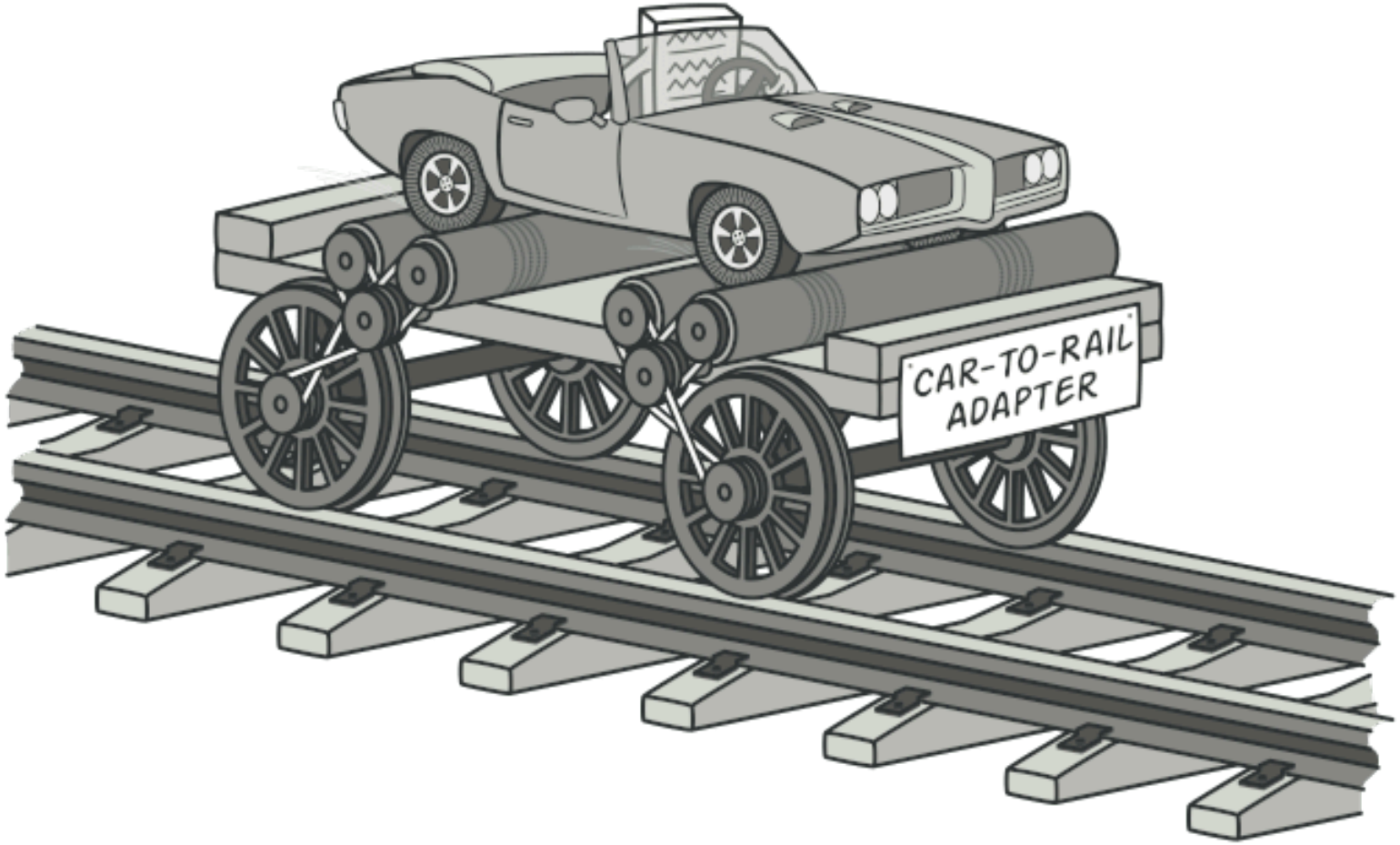


Définition :

- une technique "red/green/refactor"
- une façon de faire des "petits pas"
- une façon de toujours savoir exactement ce qui marche ou pas
- une façon de produire une "doc" de comment utiliser le code
- une façon d'avoir du code testable
- une façon d'avoir du feedback rapide
- une façon d'être certain d'avoir des tests à la fin
- une forme de "spécification exécutable"
- une technique de design ("test-driven design" ?)
- n'est PAS la solution ultime à tous les problèmes

Mise en pratique tout-à-l'heure !

N°7



Qu'est-ce que vous en pensez ?

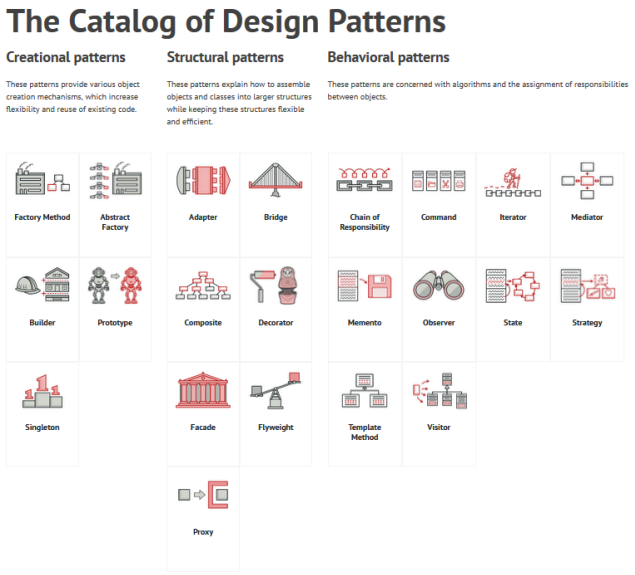
Quel terme j'essaye de vous faire deviner ?

DESIGN PATTERNS

Design Pattern : un pattern ("motif") d'une même solution pour un même problème.

chat = "animal quadrupède à fourrure, qui fait miaou!"

catalogue de problèmes/solutions connus



N°8

Dites "panier piano" 10 fois rapidement !

Qui l'a fait le + vite correctement ?

Qui le ferait encore mieux ?

```
for _ in range(10):
    print("panier, piano")
```

Quel terme j'essaye de vous faire deviner ?

CI/CD

Continuous Integration (CI) : "intégration en continu", les changements de code sont pris en compte incrémentalement, et l'application est reconstruite/retestée/re... à chaque fois

Pas de notion de branche git ...

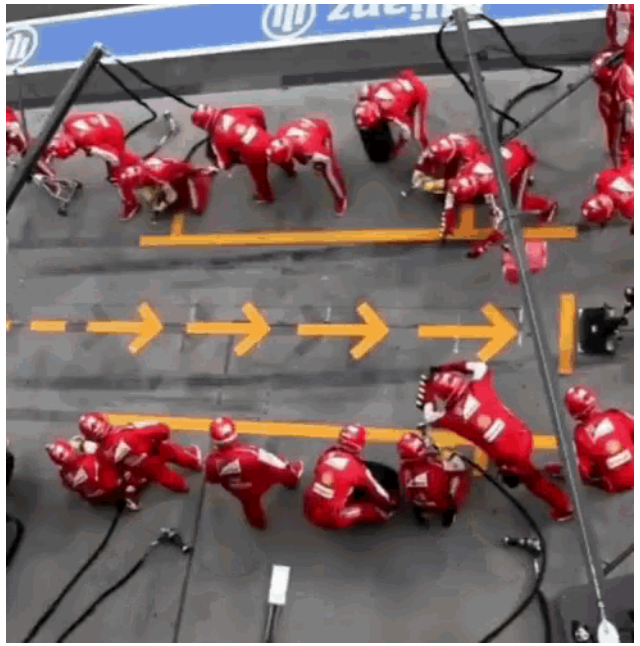
Trunk versus feature branches

Continuous Delivery (CD) : livraison en continu, les changements sont livrés, prêts à être utilisés.

Continuous Deployment (CD) : déploiement en continu, les changements sont automatiquement déployés et utilisés.

Ensemble d'outils et leurs pratiques associées

N°9



Qu'est-ce que c'est ?

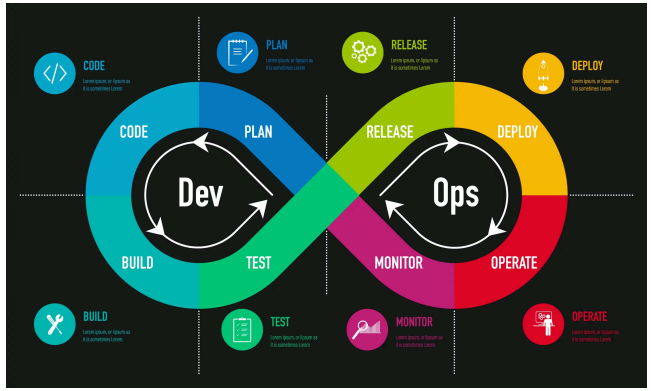
Comment il en sont arrivés à ce niveau d'efficacité ?

Comment combiner rapidité et fiabilité ?

Quel terme j'essaye de vous faire deviner ?

DEVOPS

DevOps : ensemble de pratiques, outils, et changements culturels pour accorder la production de code avec son exécution en prod.



Dérive corporate :

DevOps : non pas une culture d'avoir un objectif commun, mais juste le rôle des gens qui servent de tampon entre les Dev et les Ops, ou bien qui font le boulot des deux.

Shift Left et Dev____Ops

N°10

Sortez vos mains, on va faire un atelier !

Faites-le 5 fois d'affilée.

Qui pense maîtriser ?

Quel terme j'essaye de vous faire deviner ?

KATA / DOJO

Kata : dans les arts martiaux japonais, suite codée de mouvements constituant un exercice d'entraînement à la pureté du geste.
— Le Robert

Dojo : salle où l'on pratique les arts martiaux.
— Le Robert

Le dōjō est le lieu où l'on étudie/cherche la voie.
[...]
Le dojo est un lieu où l'on progresse. Cette progression est obligatoirement supervisée et contrôlée par un maître.
— Wikipedia

Practice makes perfect.

N°11

Question pour un (groupe) champion !

Top !

- je suis un concept de programmation
- je suis issu de l'orienté-objet
- je promets du code maintenable, testable, flexible, fiable, découplé, ...
- je suis composé de 5 lettres
- je suis un acronyme
- je suis synonyme de robuste
- je suis ...
- je suis ...

SOLID !

SOLID

- SRP : Single Responsibility Principle
  - une classe ne doit faire qu'une seule chose, et le faire bien (pour qu'elle n'ait qu'une seule raison de changer)
- OCP : Open-Closed Principle
  - une classe est ouverte aux extensions, mais fermées aux modifications
- LSP : Liskov Substitution Principle
  - si une méthode prend en paramètre (ou renvoie) un Animal, alors ça peut concrètement être un Cat ou un Dog car ce sont des Animal
- ISP : Interface Segregation Principle
  - une interface ne doit contenir que ce qu'il faut pour ses clients
- DIP : Dependency Inversion Principle
  - le code doit dépendre de classes abstraites (qui lui sont fournies) et non pas de classes concrètes (qu'il instancie)

N°12

Petite anecdote de complexité de cache ...

Quel terme j'essaye de vous faire deviner ?

KISS

KISS : keep it simple stupid !

Conserve la simplicité idiote !

C'est compliqué !

Je n’ai fait [cette lettre] plus longue que parce que je n’ai pas eu le loisir de la faire plus courte.
— Blaise Pascal

YAGNI : You ain't gonna need it

T'en auras pas b'soin

N°13

```c
main(argc, argv)
char argv[][]; {
    // [...]
    char tmp0[], tmp1[], tmp2[], tmp3[];
    char glotch[100][], clist[50][], llist[50][], ts[500];
    char tsp[], av[50][], t[];
    auto nc, nl, cflag, i, j, c;

    tmp0 = tmp1 = tmp2 = tmp3 = "//";
    tsp = ts;
    i = nc = nl = cflag = 0;
    while(++i < argc) {
        if(*argv[i] == '-' & argv[i][1]=='c')
            cflag++;
        else {
            t = copy(argv[i]);
            if((c=getsuf(t))=='c') {
                clist[nc++] = t;
                llist[nl++] = setsuf(copy(t));
            } else {
            if (nodup(llist, t))
                llist[nl++] = t;
            }
        }
    }
    if(nc==0)
        goto nocom;
    // [...]
```

Que pensez-vous de ce code ?

Quel terme j'essaye de vous faire deviner ?

CODE LEGACY

Impossible de proposer une définition !

- ou bien du code écrit il y a très longtemps (à jour ou pas)
- ou bien du code plutôt récent, mais qui a beaucoup de dépendances qui ne sont pas à jour
- ou bien du code écrit par d'autres personnes (peu importe quand), qu'on a récupéré mais qu'on aurait pas écrit comme ça
- ou bien du code écrit par soi, mais on ne l'écrirait plus pareil maintenant
- ou bien du code sans doc
- ou bien du code sans test
- ou bien du code tellement mal conçu (dette technique) qu'on n'arrive rien à faire sans causer des régressions
- ou bien un système qu'on ne comprend pas car peu clair
- ou bien un système qu'on essaye de remplacer (rewrite)
- ou bien tout à la fois

Jamais un compliment, surtout synonyme de problèmes.

N°14

```
float mysterious_function(float a){
    float b = 0.5f * a;
    int c = *(int*)&a;
    c = 0x5f3759df - (c >> 1);
    a = *(float*)&c;
    a = a*(1.5f - b*a*a);
    return a;
}
```

C'est quoi cette fonction ? (si vous ne la connaissez pas déjà)

Que fait la lib Python "ftfy" ?

Que fait la lib Python "PyERFA" ?

Quel terme j'essaye de vous faire deviner ?

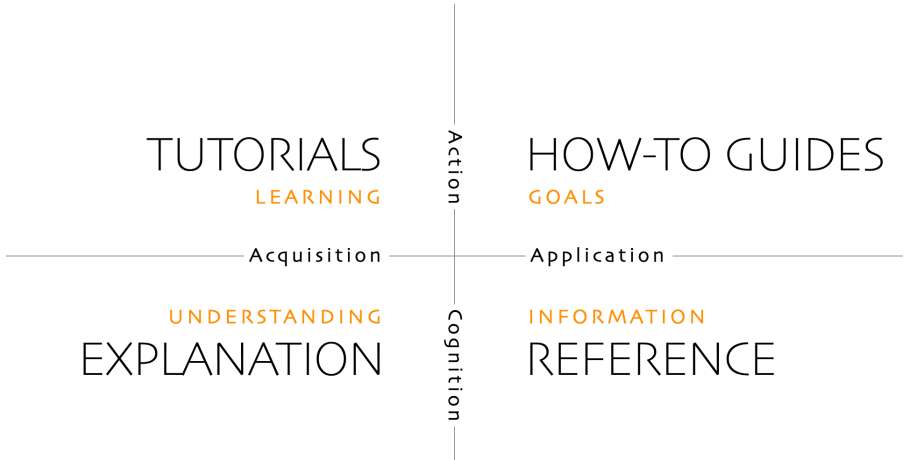
DOCUMENTATION

Très difficile à définir (même pour un dico !)

Produire des documents pour expliquer quelque chose :

- quel type de document ?
- à quel but ?
- pour quoi ?
- comment ?
- quand ?
- stocké où ?
- ...

Philosophie (Diátaxis) et outillage



N°15 : ARCHITECTURE

Architecture : choisir quelles portes on veut s'ouvrir, lesquelles on va se fermer, et les coûts/bénéfices associés.

monolith

microservices

Architectures :

- Héxagonale / Clean / Onion
- en couche
- Orientée-services
- Micro-services
- CQRS-ES
- ...

N°16 : DDD

DDD: Domain-driven design, mettre le "métier" au centre des discussions, du code, des pratiques.

Un sujet très large.

RÉCAPITULATIF

- Code review
- code smell
- tests
- refactoring
- pair/mob programming/software teaming
- TDD
- Design patterns
- CI/CD
- Dev(Sec)Ops
- dojo + kata
- SOLID
- KISS / YAGNI
- Legacy
- Documentation
- Archi héxa/clean/couches/orientée-services/micro-services/cqrs-es
- DDD

POUR CONTINUER

Quelles définitions pour :

- couplage ?
- dette technique ?
- Lean ?
- mock ?

Pour continuer à apprendre :

- conférences et meetups Craft
- replays des conférences
- podcasts, livres, articles, ...
- pratiquer
- le livre "Software Craft" (Dunod)